> The examples in this guide use the Toolkit DSL. See Toolkit DSL for an introduction.

This guide explains how to build maintainable terminal applications using the Model-View-Controller (MVC) pattern with TamboUI.

Architecture Overview

The recommended architecture separates concerns into three layers:

```
┌──────────────┐      reads      ┌──────────────┐
│  Controller  │ ◄───────────────│    View      │
│   (State)    │                 │  (Element)   │
└──────────────┘                 └──────────────┘
       ▲                                 │
       │         dispatches              │
       └─────────────────────────────────┘
                 events
```

- **Controller**: Holds all application state and provides methods to modify it
- **View**: A pure function that reads state from the controller and returns an `Element`
- **Events**: Dispatched to the controller, which updates state

The Controller

The controller is a plain Java class that encapsulates your application state:

```java
public class TodoController {
    private final List<TodoItem> items = new ArrayList<>();
    private final TextInputState inputState = new TextInputState();
```

```java
    private int selectedIndex = 0;
    private boolean inputMode = false;

    public record TodoItem(String text, boolean done) {}

    // Queries (read state)
    public List<TodoItem> items() { return List.copyOf(items); }
    public int selectedIndex() { return selectedIndex; }
    public TextInputState inputState() { return inputState; }
    public boolean isInputMode() { return inputMode; }

    // Commands (modify state)
    public void moveUp() {
        if (selectedIndex > 0) selectedIndex--;
    }

    public void moveDown() {
        if (selectedIndex < items.size() - 1) {
            selectedIndex++;
        }
    }

    public void toggleSelected() {
        if (!items.isEmpty()) {
            var item = items.get(selectedIndex);
            items.set(selectedIndex, new TodoItem(item.text(), !item.done()));
        }
    }

    public void deleteSelected() {
        if (!items.isEmpty()) {
            items.remove(selectedIndex);
            if (selectedIndex >= items.size() && selectedIndex > 0) {
                selectedIndex--;
            }
        }
    }

    public void startInput() { inputMode = true; }
    public void cancelInput() { inputMode = false; inputState.clear(); }

    public void submitInput() {
        if (inputState.length() > 0) {
            items.add(new TodoItem(inputState.text(), false));
            selectedIndex = items.size() - 1;
        }
        inputMode = false;
```

```
            inputState.clear();
        }
    }
```

Controller Design Principles

- **Queries** are pure read-only methods that return state
- **Commands** modify state and have no return value
- Return defensive copies of collections (`List.copyOf()`)
- Keep state private, expose through methods

The View

The view is a pure function that transforms controller state into UI elements:

```
public class TodoView {
    private final TodoController controller;

    public TodoView(TodoController controller) {
        this.controller = controller;
    }

    public Panel render() {
        return panel("Todo List",
            renderList(),
            spacer(),
            renderInput(),
            renderHelp()
        ).rounded().id("main").focusable();
    }

    private Element renderList() {
        var items = controller.items();
        if (items.isEmpty()) {
            return text("No items. Press 'a' to add one.").dim().italic();
        }

        var elements = new Element[items.size()];
        for (int i = 0; i < items.size(); i++) {
```

```
            elements[i] = renderItem(i, items.get(i));
        }
        return column(elements);
    }

    private Element renderItem(int index, TodoController.TodoItem item) {
        var checkbox = item.done() ? "[x]" : "[ ]";
        var element = text(checkbox + " " + item.text());

        if (index == controller.selectedIndex()) {
            element = element.reversed();
        }
        if (item.done()) {
            element = element.dim().crossedOut();
        }
        return element;
    }

    private Element renderInput() {
        if (!controller.isInputMode()) {
            return text("");
        }
        return row(
            text("New: ").cyan(),
            textInput(controller.inputState()).fill()
        );
    }

    private Element renderHelp() {
        if (controller.isInputMode()) {
            return text("[Enter] Save  [Esc] Cancel").dim();
        }
        return text("[a] Add  [Space] Toggle  [d] Delete  [j/k] Navigate  [q] Quit
    }
}
```

View Design Principles

- Views are **pure functions**: `Controller → Element`
- No side effects in render methods
- All state comes from the controller
- Conditional rendering based on controller state

Event Handling

Using Lambda Handlers

For simple applications, attach handlers directly:

```
TodoController controller = new TodoController();
TodoView view = new TodoView(controller);

try (var runner = ToolkitRunner.create()) {
    runner.run(() ->
        view.render()
            .onKeyEvent(event -> handleEvent(event, controller))
    );
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

```
private static EventResult handleEvent(KeyEvent event, TodoController ctrl) {
    if (ctrl.isInputMode()) {
        return handleInputMode(event, ctrl);
    }
    return handleNormalMode(event, ctrl);
}

private static EventResult handleInputMode(KeyEvent event, TodoController ctrl) {
    if (event.isCancel()) {
        ctrl.cancelInput();
        return EventResult.HANDLED;
    }
    if (event.isSelect()) {
        ctrl.submitInput();
        return EventResult.HANDLED;
    }
    if (handleTextInputKey(ctrl.inputState(), event)) {
        return EventResult.HANDLED;
    }
    return EventResult.UNHANDLED;
}

private static EventResult handleNormalMode(KeyEvent event, TodoController ctrl) {
```

```
    if (event.isChar('a')) { ctrl.startInput(); return EventResult.HANDLED; }
    if (event.isUp()) { ctrl.moveUp(); return EventResult.HANDLED; }
    if (event.isDown()) { ctrl.moveDown(); return EventResult.HANDLED; }
    if (event.isSelect()) { ctrl.toggleSelected(); return EventResult.HANDLED; }
    if (event.isChar('d')) { ctrl.deleteSelected(); return EventResult.HANDLED; }
    return EventResult.UNHANDLED;
}
```

Separate Key Handler Class

For complex applications, extract event handling:

```
public class TodoKeyHandler {
    private final TodoController controller;

    public TodoKeyHandler(TodoController controller) {
        this.controller = controller;
    }

    public EventResult handle(KeyEvent event) {
        if (controller.isInputMode()) {
            return handleInputMode(event);
        }
        return handleNormalMode(event);
    }

    private EventResult handleInputMode(KeyEvent event) {
        if (event.isCancel()) {
            controller.cancelInput();
            return EventResult.HANDLED;
        }
        if (event.isConfirm()) {
            controller.submitInput();
            return EventResult.HANDLED;
        }
        // ... handle text input
        return EventResult.UNHANDLED;
    }

    private EventResult handleNormalMode(KeyEvent event) {
        if (event.isChar('a')) {
            controller.startInput();
            return EventResult.HANDLED;
        }
```

```
        // ... more handlers
        return EventResult.UNHANDLED;
    }
}
```

Event Result

Return values control event propagation:

- `EventResult.HANDLED` - Event was processed, stop propagation
- `EventResult.UNHANDLED` - Event was not processed, continue routing

Focus Management

Making Elements Focusable

Both `id()` AND `focusable()` are required:

```
panel("Panel A")
    .id("panel-a")          // Unique identifier (REQUIRED)
    .focusable()            // Enable focus (REQUIRED)
    .borderColor(Color.GRAY)
    .focusedBorderColor(Color.CYAN);  // Visual feedback
```

Focus Navigation

Key	Action
Tab	Focus next element
Shift+Tab	Focus previous element
Mouse click	Focus clicked element

Key	Action
ESC	Clear focus (if no element handles it)

Multi-Panel Applications

```
column(
    panel("Settings")
        .id("settings")
        .focusable()
        .onKeyEvent(event -> handleSettingsKey(event, controller)),

    panel("Actions")
        .id("actions")
        .focusable()
        .onKeyEvent(event -> handleActionsKey(event, controller))
);
```

Dialogs and Overlays

The Dialog Element

```
var inputDialog = dialog("New Directory",
    text("Enter name:"),
    textInput(inputState),
    text("[Enter] Confirm  [Esc] Cancel").dim()
).rounded()
 .borderColor(Color.CYAN)
 .width(50)
 .onConfirm(() -> createDirectory(inputState.text()))
 .onCancel(() -> dismissDialog());
```

Implementing Custom Overlay

For custom dialogs, implement `Element` directly:

```java
public class MyView implements Element {
    private final MyController controller;

    public MyView(MyController controller) {
        this.controller = controller;
    }



    @Override
    public void render(Frame frame, Rect area, RenderContext context) {
        // 1. Render main UI
        var ui = column(header(), content(), footer());
        ui.render(frame, area, context);

        // 2. Render dialog on top (if present)
        if (controller.hasDialog()) {
            renderDialog(frame, area, context);
        }
    }

    private void renderDialog(Frame frame, Rect area, RenderContext context) {
        int dialogWidth = 50;
        int dialogHeight = 6;
        int x = (area.width() - dialogWidth) / 2;
        int y = (area.height() - dialogHeight) / 2;
        var dialogArea = new Rect(area.x() + x, area.y() + y, dialogWidth, dialogH

        // Clear area first
        frame.renderWidget(Clear.INSTANCE, dialogArea);

        // Render dialog panel
        var dialog = panel("Confirm",
            text(controller.dialogMessage()),
            text("[y] Yes  [n] No  [Esc] Cancel").dim()
        ).rounded().borderColor(Color.YELLOW);

        dialog.render(frame, dialogArea, context);
    }

    @Override
    public EventResult handleKeyEvent(KeyEvent event, boolean focused) {
        // ESC should dismiss dialog, not clear focus
        if (event.isCancel() && controller.hasDialog()) {
            controller.dismissDialog();
            return EventResult.HANDLED;
```

```
        }
        // ... other handling
        return EventResult.UNHANDLED;
    }

    @Override
    public Size preferredSize(int availableWidth, int availableHeight, RenderConte
        return Size.UNKNOWN;
    }

    private Element header() { return text("Header"); }
    private Element content() { return text("Content"); }
    private Element footer() { return text("Footer"); }
}
```

Complete Example

```
public static class TodoApp {

    // ====================================================================
    // CONTROLLER
    // ====================================================================

    static class Controller {
        public record Item(String text, boolean done) {}

        private final List<Item> items = new ArrayList<>();
        private final StringBuilder input = new StringBuilder();
        private int cursor = 0;
        private boolean editing = false;

        public List<Item> items() { return List.copyOf(items); }
        public int cursor() { return cursor; }
        public String input() { return input.toString(); }
        public boolean isEditing() { return editing; }
        public boolean isEmpty() { return items.isEmpty(); }

        public void cursorUp() { if (cursor > 0) cursor--; }
        public void cursorDown() { if (cursor < items.size() - 1) cursor++; }

        public void toggle() {
```

```
            if (!items.isEmpty()) {
                var item = items.get(cursor);
                items.set(cursor, new Item(item.text(), !item.done()));
            }
        }

        public void delete() {
            if (!items.isEmpty()) {
                items.remove(cursor);
                if (cursor >= items.size() && cursor > 0) cursor--;
            }
        }

        public void startEditing() { editing = true; }
        public void cancelEditing() { editing = false; input.setLength(0); }
        public void typeChar(char c) { input.append(c); }
        public void backspace() {
            if (input.length() > 0) input.setLength(input.length() - 1);
        }

        public void submit() {
            if (input.length() > 0) {
                items.add(new Item(input.toString(), false));
                cursor = items.size() - 1;
            }
            editing = false;
            input.setLength(0);
        }
    }

    // ======================================================================
    // VIEW
    // ======================================================================

    static class View {
        private final Controller ctrl;

        View(Controller ctrl) { this.ctrl = ctrl; }

        Panel render() {
            return panel("Todo List",
                ctrl.isEmpty() ? emptyState() : itemList(),
                spacer(),
                inputArea(),
                helpBar()
            ).rounded().borderColor(Color.DARK_GRAY)
             .focusedBorderColor(Color.CYAN)
```

```
            .id("main").focusable();
        }

        private Element emptyState() {
            return text("No items yet. Press 'a' to add one.").dim().italic();
        }

        private Element itemList() {
            var items = ctrl.items();
            var elements = new Element[items.size()];
            for (int i = 0; i < items.size(); i++) {
                elements[i] = itemRow(i, items.get(i));
            }
            return column(elements);
        }

        private Element itemRow(int index, Controller.Item item) {
            var prefix = item.done() ? "[x] " : "[ ] ";
            var elem = text(prefix + item.text());

            if (index == ctrl.cursor()) elem = elem.reversed();
            if (item.done()) elem = elem.dim().crossedOut();

            return elem;
        }

        private Element inputArea() {
            if (!ctrl.isEditing()) return text("");
            return row(
                text("New: ").cyan(),
                text(ctrl.input() + "_").bold()
            );
        }

        private Element helpBar() {
            var help = ctrl.isEditing()
                ? "[Enter] Save  [Esc] Cancel"
                : "[a] Add  [Space] Toggle  [d] Delete  [j/k] Move  [q] Quit";
            return text(help).dim();
        }
    }

    // ====================================================================
    // EVENT HANDLER
    // ====================================================================

    static EventResult handleKey(KeyEvent event, Controller ctrl) {
```

```
        if (ctrl.isEditing()) {
            if (event.isCancel()) { ctrl.cancelEditing(); return EventResult.HANDL
            if (event.isSelect()) { ctrl.submit(); return EventResult.HANDLED; }
            if (event.code() == KeyCode.CHAR && event.character() >= 32) {
                ctrl.typeChar(event.character());
                return EventResult.HANDLED;
            }
            if (event.code() == KeyCode.BACKSPACE) {
                ctrl.backspace();
                return EventResult.HANDLED;
            }
            return EventResult.UNHANDLED;
        }

        if (event.isChar('a')) { ctrl.startEditing(); return EventResult.HANDLED;
        if (event.isUp()) { ctrl.cursorUp(); return EventResult.HANDLED; }
        if (event.isDown()) { ctrl.cursorDown(); return EventResult.HANDLED; }
        if (event.isSelect()) { ctrl.toggle(); return EventResult.HANDLED; }
        if (event.isChar('d')) { ctrl.delete(); return EventResult.HANDLED; }
        return EventResult.UNHANDLED;
    }

    // ====================================================================
    // MAIN
    // ====================================================================

    public static void main(String[] args) throws Exception {
        var controller = new Controller();
        var view = new View(controller);

        // Add sample data
        controller.items.add(new Controller.Item("Learn TamboUI", false));
        controller.items.add(new Controller.Item("Build awesome apps", false));

        try (var runner = ToolkitRunner.create()) {
            runner.run(() ->
                view.render()
                    .onKeyEvent(e -> handleKey(e, controller))
            );
        }
    }
}
```

Benefits of This Architecture

Benefit	Description
Testable	Controller can be unit tested without any UI
Reusable	Same controller can power different views
Predictable	State changes only through controller methods
Debuggable	Easy to log/inspect state transitions
Maintainable	Clear separation of concerns

Next Steps

- Explore the Widgets Reference for available components
- Learn about Bindings and Actions for handling user input
- Read the Developer Guide for creating custom widgets
- See the API Levels for alternative approaches

PREVIOUS
Bindings and Actions

NEXT
Developer Guide